

Lecture 1: (Artificial) Intelligence, Learning, Language, and Meaning

Language models now write programs, prove small lemmas, summarize papers, and hold conversations that many people find hard to tell apart from human ones. How these systems work is mostly well understood at the level of their parts: a probability distribution over strings, parameterized by a large network, fitted to text, adjusted with demonstrations and rewards, and sampled from at inference time. Understanding why they work, what they can and cannot do, and how to reason about them is much less settled. This course builds the whole stack from first principles, and it does so from the viewpoint of a theoretical computer scientist.

This first lecture has three parts. It begins with the course itself: its scope, its eight weeks, and the questions it keeps asking. It then steps back to three old questions that the systems of this course make newly concrete: what intelligence is, what it means to learn, and what language and meaning are. Those sections are more discursive than the rest of the course; they are meant to frame what follows, not to settle anything. The lecture ends by fixing the terminology of artificial intelligence, machine learning, and deep learning, reviewing the basic mathematics that later lectures assume, and pointing to the supplementary material where each basic topic is developed in full.

1. The course and its point of view

The central object

A generative model produces structured objects: a paragraph, a program, an image, a sequence of actions. This course concentrates on language, for two reasons. Language models are the systems that changed the field between 2018 and today, and language is the setting in which the mathematics is richest: it has discrete symbols, recursive structure, compositional meaning, dependence on context, and a long history in logic, linguistics, and the theory of computation.

The object at the center of the course is a probability distribution over strings. Fix a finite vocabulary $\mathcal{V}$ of tokens (characters, words, or pieces of words) together with a special end-of-sequence token EOS. A language model assigns a probability to every finite string $x_1x_2\cdots x_T$ by predicting one token at a time:

$$q_\theta(x_{1:T}, \text{EOS}) = \prod_{t=1}^{T+1} q_\theta(x_t \mid x_{<t}), \quad x_{T+1} = \text{EOS}.$$

Here $x_{<t}$ denotes the tokens before position t , and θ collects the model's parameters. The identity on the right is just the chain rule of probability; the modeling content lies entirely in how each conditional distribution $q_\theta(\cdot \mid x_{<t})$ is computed and how θ is chosen. Almost everything in the course is a statement about this object. Weeks 2 and 3 are about how the conditionals are computed: first with counts and grammars, then with attention. Week 4 is about how θ is fitted at scale. Weeks 5 through 7 are about how the fitted distribution is changed so that sampling from it produces a helpful assistant rather than a plausible continuation of web text. Week 8 is about how the distribution is used: decoded, searched, served, and evaluated.

The eight weeks

Week	Lecture	Central development
1	(Artificial) Intelligence, Learning, Language, and Meaning	The course and its questions; intelligence, learning, and language; terminology and basics of AI, ML, and DL
2	The Traditional NLP Approach to Language Modeling	Formal languages and stochastic sources; n-gram models, smoothing, and perplexity; hidden Markov models; distributional semantics; the first neural language models
3	The Modern Approach: Attention and Transformers	Recurrent networks and their limits; attention; the complete Transformer computation, its costs, and what it can represent

Week	Lecture	Central development
4	Scale and Pretraining	Tokenization, data, objectives, optimization, scaling laws, and training systems
5	From Base Model to Assistant: the ChatGPT Revolution	In-context learning; instruction following; the ideas behind Instruct-GPT and ChatGPT
6	Reinforcement Learning and Sequence Generation	Bandits, MDPs, Bellman equations, policy gradients, PPO; RL when the actions are tokens
7	Post-training in the LLM Stack: SFT and RLHF	Supervised finetuning, reward models, RLHF, DPO, and verifiable rewards
8	Inference	Decoding, test-time computation, serving, and evaluation

The order follows the dependencies between ideas. Traditional language modeling comes first because the questions it raises (sparsity, evaluation, what a distribution over strings is) remain the questions of the neural era. Attention comes before scale because scaling is a statement about a fixed architecture. The general theory of reinforcement learning gets a lecture of its own before its use in post-training, because the post-training objectives are much easier to understand once the underlying decision problem is clear.

The point of view

The course is taught from the viewpoint of a theoretical computer scientist. That viewpoint is less a set of results than a habit of asking particular questions:

- **What is computed?** A trained Transformer computes a specific function; decoding computes another; a chat system wraps both in a larger program. Being precise about the function is the first step to reasoning about it.
- **With what resources?** Training compute, memory, the cost of one forward pass, and the cost of generating a complete answer are different quantities, and they scale differently.
- **From what information?** A learner can only extract what its data contain. Much of the course is about which information each stage of training supplies: raw text, demonstrations, comparisons, or checkable outcomes.
- **With what guarantee?** Some claims about language models are theorems, some are measurements, and some are conjectures supported by a handful of experiments. Keeping them apart is half the work.

A recurring instance of this habit is the separation of **expressivity**, **optimization**, and **generalization**. A model class may be able to express a computation without any efficient procedure finding it; a procedure may find parameters that fit the training data without the result working on new inputs. Much confusion about what language models “can do” comes from running these three questions together.

2. Intelligence

Turing’s question

In 1950 Alan Turing proposed to replace the question “Can machines think?”, which he considered too ill-defined to discuss, with a game. An interrogator converses by text with two hidden parties, a person and a machine, and must decide which is which. The machine does well if the interrogator is wrong about as often as with two people. Turing predicted that by about the year 2000 a machine with roughly 10^9 bits of storage would play well enough that an average interrogator would have no more than a 70 percent chance of a correct identification after five minutes. Turing, *Computing Machinery and Intelligence*.

Two features of the paper matter for this course. First, the test is behavioral and linguistic: intelligence is to be judged by conversation, which is exactly the capability that language models now exhibit. Second, most of the paper is not about the game at all but about objections to machine intelligence, and its final section, “Learning Machines,” argues that the practical route to a thinking machine is not to program adult competence directly. “Instead of trying to produce a programme to simulate the adult mind, why not rather try to produce one which simulates the child’s?” The child machine would then be

educated. Turing also answers Lady Lovelace’s objection that a machine “has no pretensions to originate anything” and can only do what it is told, by observing that a learning machine’s behavior is not fully anticipated by its programmer. Seventy years later, the systems that come closest to passing his test were built in roughly the way he suggested: a general learning procedure, a great deal of experience, and comparatively little explicit programming.

The imitation game is a poor definition of intelligence, and Turing did not offer it as one. It is anthropocentric, it rewards deception, and it can be passed on short conversations by systems with very limited abilities, as Joseph Weizenbaum’s 1966 ELIZA program showed with a few pattern-matching rules. Its lasting value is methodological: it replaces a metaphysical question with an operational one about observable behavior. Much of this course does the same thing at a smaller scale, replacing “does the model understand?” with “what does the model compute, and how does it perform on a specified distribution of tasks?”

Symbols, networks, and the Dartmouth conjecture

The 1955 proposal for the Dartmouth summer workshop, which named the field, stated its founding conjecture: “every aspect of learning or any other feature of intelligence can in principle be so precisely described that a machine can be made to simulate it.” Its list of topics included how a computer can be programmed to use a language, neuron nets, self-improvement, abstraction, and randomness and creativity. McCarthy, Minsky, Rochester, and Shannon, Dartmouth proposal. Almost every item on that list is a topic of this course.

Two broad programs developed from it. The symbolic program held that intelligence consists in manipulating symbols according to rules. Allen Newell and Herbert Simon stated its strongest form as the physical symbol system hypothesis: “a physical symbol system has the necessary and sufficient means for general intelligent action.” Newell and Simon, 1976. Theorem provers, planners, expert systems, and hand-written grammars belong to this program. The connectionist program held that intelligence emerges from large numbers of simple units adjusting their connections. It begins with Warren McCulloch and Walter Pitts’s 1943 logical model of neurons, which showed that networks of threshold units can compute any Boolean function, and with Frank Rosenblatt’s perceptron, a single threshold unit that learns its weights from labeled examples. McCulloch and Pitts, 1943; Rosenblatt, 1958. Marvin Minsky and Seymour Papert’s 1969 book *Perceptrons* proved that single-layer perceptrons cannot compute predicates such as parity or connectedness from local features, and the lack of an effective training method for multilayer networks stalled the program until backpropagation was popularized in the 1980s.

A modern language model is a descendant of the connectionist program: a network of simple units whose connections are learned. Yet it operates on discrete symbols, produces symbol sequences, and can be prompted to carry out symbolic procedures such as arithmetic or program execution, sometimes reliably and sometimes not. Whether it manipulates symbols in the sense of Newell and Simon, and whether that question even has a sharp answer, is one of the live debates this course equips you to think about.

Definitions and measures

There is no agreed definition of intelligence, but there are useful families of definitions. The standard textbook of the field, Stuart Russell and Peter Norvig’s *Artificial Intelligence: A Modern Approach*, sorts them along two axes: whether they concern thinking or acting, and whether they measure against human performance or against an ideal of rationality. The book itself adopts the rational-agent view: an agent perceives its environment and acts so as to maximize the expected value of a performance measure, given what it has perceived. This view is convenient for engineering because it turns intelligence into an optimization problem, and it is the view behind reinforcement learning in Week 6.

Shane Legg and Marcus Hutter pushed the rational-agent view to a formal limit. They define the universal intelligence of an agent π as its expected performance across all computable environments, weighted by simplicity:

$$\Upsilon(\pi) = \sum_{\mu \in E} 2^{-K(\mu)} V_{\mu}^{\pi}.$$

Here E is the class of computable reward-generating environments, V_{μ}^{π} is the expected total reward the agent obtains in environment μ , and $K(\mu)$ is the Kolmogorov complexity of μ : the length of the shortest program that implements it on a fixed universal machine. Legg and Hutter, 2007. The definition is elegant and explains a common intuition, that intelligence is the ability to do well across many environments

with a bias toward simple ones. It is also incomputable, since K is incomputable and the sum ranges over all computable environments, so it cannot be used as a benchmark directly.

François Chollet objects that measuring skill at tasks, even many tasks, conflates intelligence with the knowledge and priors that were used to build the skill. A system can buy performance on any fixed set of tasks with enough training data. He proposes instead to measure intelligence as *skill-acquisition efficiency*: how quickly a system acquires new skills over a scope of tasks, relative to its priors, its experience, and the difficulty of generalization. Chollet, 2019. The distinction matters for language models, which are trained on a large fraction of the written record: their performance on a benchmark says something about their skill, but by itself very little about how efficiently they learn.

Behavior, understanding, and the bitter lesson

John Searle’s Chinese room argument attacks the behavioral view directly. A person who speaks no Chinese sits in a room with a rulebook for manipulating Chinese characters. Questions in Chinese come in; by following the rules, the person sends out answers that native speakers find perfectly sensible. Searle argues that the person does not understand Chinese, that neither does the room, and therefore that running the right program is not sufficient for understanding. Searle, 1980. The argument has many replies, the most common being that the system as a whole, rather than the person, is the relevant candidate for understanding. Its relevance here is that language models are almost exactly the room: a fixed procedure mapping input strings to output strings, with competence derived from the statistics of text rather than from contact with the world. Section 4 returns to this question in the form it takes for language models: can meaning be learned from form alone?

Two further observations from the history of AI shape the course. The first is Moravec’s paradox, named after Hans Moravec: it proved comparatively easy to make computers perform well at tasks that humans find intellectually demanding, such as chess or symbolic integration, and very hard to give them the perception and mobility of a one-year-old child. The skills that seem hard to us are recent in evolutionary terms and small in their demands; the skills that seem effortless are old and enormous. Language models add a twist: fluent language, which humans find effortless, turned out to be learnable from text at scale, while reliable multistep reasoning, which humans find effortful, remains harder for them.

The second is Richard Sutton’s “bitter lesson”: across seventy years of AI research, general methods that exploit increasing computation, chiefly search and learning, have eventually beaten methods that build in human knowledge about the domain, often by a large margin. Sutton, 2019. The progression from hand-written grammars to n-gram models to neural language models trained on trillions of tokens is one of the clearest instances. The lesson is empirical, not a theorem, and it does not say that structure is useless: the Transformer is itself a carefully designed structure, and much of Weeks 3 and 4 is about which structural choices matter at scale.

3. Learning

The problem of induction

Every learning system generalizes from what it has seen to what it has not. David Hume observed in 1748 that no argument from experience can establish that unobserved cases will resemble observed ones without assuming the very regularity it sets out to prove. Hume, *An Enquiry Concerning Human Understanding*. Any justification of induction must therefore rest on assumptions about the world that the data alone cannot supply.

Machine learning has a precise version of this observation, usually called a no-free-lunch theorem. Consider binary classification on a finite domain of N points, and suppose the true labeling is chosen uniformly at random from all 2^N functions. After observing the labels of any subset of the points, the label of every unobserved point is still a fair coin, independent of everything seen. Every learning algorithm, however clever, then has expected error exactly one half on unseen points. Wolpert, 1996. Learning is possible only because real problems are not like this: the functions we care about are smooth, compositional, or otherwise simple, and a learner succeeds by preferring such functions. That preference is its **inductive bias**. In a language model the inductive bias lives in the tokenizer, the architecture, the optimizer, the order of the data, and the objective, and much of the course is about what each of these contributes.

Learning as a computational problem

Leslie Valiant turned learning into a problem about algorithms and resources. In the probably approximately correct (PAC) model, there is an unknown distribution $\mathcal{D}$ over an input domain X and an unknown target concept $c : X \rightarrow \{0, 1\}$ drawn from a known class $\mathcal{C}$. The learner receives m independent examples $(x_i, c(x_i))$ with $x_i \sim \mathcal{D}$ and must output a hypothesis h whose error

$$\text{err}_{\mathcal{D}}(h) = \Pr_{x \sim \mathcal{D}}[h(x) \neq c(x)]$$

is at most ε , with probability at least $1 - \delta$ over the sample. The class is efficiently PAC learnable if some algorithm does this for every $\mathcal{D}$ and every $c \in \mathcal{C}$, using a number of examples and an amount of time polynomial in $1/\varepsilon$, $1/\delta$, and the size of the problem. Valiant, *A Theory of the Learnable*, 1984.

The definition makes both halves of Hume's problem explicit. The distribution $\mathcal{D}$ is the assumption that the future resembles the past: test inputs come from the same distribution as training inputs. The class $\mathcal{C}$ is the inductive bias: the learner knows the target is one of a restricted set of functions. Given these, learning becomes possible, and its cost can be measured.

The simplest positive result shows how. Suppose the learner searches a finite hypothesis class $\mathcal{H} \supseteq \mathcal{C}$ and outputs any hypothesis that agrees with every training example. Call a hypothesis bad if its error exceeds ε . A fixed bad hypothesis agrees with one random example with probability at most $1 - \varepsilon$, so it agrees with all m independent examples with probability at most

$$(1 - \varepsilon)^m \leq e^{-\varepsilon m}.$$

There are at most $|\mathcal{H}|$ bad hypotheses, so by the union bound the probability that any of them survives is at most $|\mathcal{H}|e^{-\varepsilon m}$. Requiring this to be at most δ gives the sample size

$$m \geq \frac{1}{\varepsilon} \left(\ln |\mathcal{H}| + \ln \frac{1}{\delta} \right).$$

This is the **Occam bound**: the number of examples needed grows with the logarithm of the number of hypotheses, that is, with the number of bits needed to name a hypothesis. For example, with $|\mathcal{H}| = 2^{20}$, $\varepsilon = 0.05$, and $\delta = 0.01$, the bound requires $m \geq 20(20 \ln 2 + \ln 100)$, about 370 examples. If $\mathcal{H}$ contains all Boolean functions on n input bits, then $\ln |\mathcal{H}| = 2^n \ln 2$, and the bound becomes exponential in n : the no-free-lunch phenomenon reappears as a sample-complexity statement. Infinite classes are handled by replacing $\ln |\mathcal{H}|$ with a combinatorial measure of complexity such as the VC dimension. The general theory, including the case where no hypothesis fits the data perfectly, is developed in Shalev-Shwartz and Ben-David, *Understanding Machine Learning* and in Chapter 3 of the course's AI and Machine Learning companion.

What efficient learning cannot do, and what queries buy

The Occam bound says nothing about time. Finding a hypothesis consistent with the data can be computationally hard even when few examples suffice, and for some classes no efficient learner exists at all under standard cryptographic assumptions. Michael Kearns and Valiant showed that deterministic finite automata cannot be efficiently PAC learned, even if the learner may output any polynomially evaluable hypothesis, unless certain cryptographic problems (such as inverting RSA) are easy. Kearns and Valiant, 1994. The intuition is that a finite automaton can compute a pseudorandom-looking function; if one could learn it from random examples, one could predict the output of a cryptographic generator.

The hardness depends on how the learner obtains information. Dana Angluin's L^* algorithm learns any regular language exactly, in time polynomial in the number of states of its minimal automaton and the length of the longest counterexample it receives, if the learner may ask two kinds of question: membership queries ("is this string in the language?") and equivalence queries ("is this automaton correct? if not, give me a counterexample"). Angluin, 1987. The contrast between the two results is a small model of a large theme in this course: what a learner can learn efficiently depends on the channel through which it gets information. Pretraining on text supplies random examples; supervised finetuning supplies chosen demonstrations; preference learning supplies comparisons; verifiable rewards supply something like answers to queries.

These results concern automata, not neural networks trained by gradient descent, and they are worst-case statements. They should be read as warnings about what cannot be guaranteed in general, not as predictions about what happens on natural language. The gap between worst-case hardness and empirical success recurs throughout the course.

Simplicity, prediction, and compression

Ray Solomonoff proposed an idealized solution to induction in 1964. Consider all programs for a fixed universal machine, and give each program p prior weight $2^{-|p|}$, so that shorter programs are more probable. The universal prior probability of a finite string x is the total weight of programs whose output begins with x . Predict the next symbol by conditioning this prior on what has been seen. Solomonoff, *A Formal Theory of Inductive Inference*, Part I, 1964. Solomonoff later proved that this predictor is optimal in a strong sense: for any computable data source μ , its total expected prediction error over an infinite sequence is bounded by a constant proportional to the complexity $K(\mu)$ of the source. Solomonoff induction is incomputable, like the Legg-Hutter measure built on it, but it makes precise an idea that runs through this course: good prediction and short description are two faces of the same thing.

The practical version of that idea is the correspondence between probability and code length. Given any probability distribution q over messages, there is a prefix code, arithmetic coding, that encodes a message x in at most $-\log_2 q(x) + 2$ bits. Conversely, by Kraft's inequality, the code lengths $\ell(x)$ of any prefix code define weights $2^{-\ell(x)}$ that sum to at most one, so every code is implicitly a probability model. If messages are drawn from a source p , the expected code length when coding with q is essentially the cross-entropy

$$H(p, q) = - \sum_x p(x) \log_2 q(x) = H(p) + D_{\text{KL}}(p \parallel q),$$

where $H(p)$ is the entropy of the source and D_{KL} is the Kullback-Leibler divergence, which is nonnegative and zero only when $q = p$. A model that predicts better compresses better, and the gap to the best possible compression is exactly the KL divergence. The minimum description length principle turns this into a method for choosing models: prefer the hypothesis H minimizing the length of H 's own description plus the length of the data coded with H 's help. Rissanen, 1978.

Training a language model by minimizing next-token log loss is, in this sense, training a compressor for text. Whether compression of the written record amounts to understanding is the question of the next section.

4. Language and meaning

What a sentence is

A written sentence is at once a sequence of marks, a grammatical construction, an expression about something, and a move in an interaction. The history of thinking about language has treated these aspects separately, and the distinctions remain useful for thinking about what a language model does and does not capture.

Rules and productivity. Pāṇini's *Aṣṭādhyāyī*, a Sanskrit grammar usually dated to the middle of the first millennium BCE, describes the forms of a language through a compact system of interacting rules, together with technical conventions for applying them. Pāṇini, *Aṣṭādhyāyī*, translated by Śrīśa Chandra Vasu, 1891. Its central insight is productivity: a finite description can determine an unbounded number of expressions, including ones never observed. Noam Chomsky made the same point the foundation of modern linguistics, describing a grammar as a finite device that generates exactly the grammatical sentences of a language, and classifying such devices by their power in what became the Chomsky hierarchy of regular, context-free, context-sensitive, and recursively enumerable languages. Chomsky, *Three Models for the Description of Language*, 1956. Week 2 develops this hierarchy and the automata that recognize each level.

Reference and composition. Aristotle's *On Interpretation* separates a name, which signifies by convention, from a statement, which can be true or false only once expressions are combined. Aristotle, *On Interpretation*. Gottlob Frege distinguished an expression's reference (what it picks out) from its sense (the way it presents what it picks out): "the author of submission 17" and "the winner of the student prize" may refer to the same person while conveying different information. Frege, *On Sense and Reference*, 1892. The meaning of a whole depends on the meanings of its parts and how they are combined. "Every student read a paper" has two readings, $\forall x \exists y R(x, y)$ and $\exists y \forall x R(x, y)$, which differ in truth conditions while using the same words. A representation that records only which words occur cannot distinguish them.

Use and implicature. Ludwig Wittgenstein's *Philosophical Investigations* examines language as a family of activities, such as giving orders, reporting, and asking, and suggests that for a large class of cases the meaning of a word is its use in the language. Wittgenstein, *Philosophical Investigations*,

1953. Paul Grice explained how speakers communicate more than they literally say. If asked whether all four experiments succeeded, the answer "Some succeeded" suggests that not all did, although it is logically consistent with all four succeeding; the suggestion can be cancelled by adding "in fact, all four did." Grice, *Logic and Conversation*, 1975. An assistant that answers questions must handle this layer of meaning, which is invisible to a purely logical analysis of sentences.

Distribution. Zellig Harris proposed that differences in meaning correlate with differences in distribution: words that occur in similar environments tend to have related meanings. Harris, *Distributional Structure*, 1954. J. R. Firth's version became a slogan: "You shall know a word by the company it keeps." Firth, 1957. Distributional evidence is exactly what a language model trained on text can see. It is informative but not the same as meaning: "increase" and "decrease" occur in very similar contexts and mean opposite things.

Learning a language from examples

Children learn their first language from a finite and noisy sample of it, mostly without explicit correction. How this is possible was one of the defining questions of twentieth-century linguistics, and it has a clean formal version from the theory of computation.

E. Mark Gold modeled language learning as follows. A class $\mathcal{L}$ of languages over an alphabet Σ is fixed, and an unknown target $L \in \mathcal{L}$ is chosen. An adversary presents a *text* for L : an infinite sequence of strings that contains every string of L and nothing outside it. After each string the learner outputs a guess, a grammar for some language in $\mathcal{L}$. The learner *identifies L in the limit* if its guesses eventually stop changing and are correct from then on; it identifies the class if it does so for every $L \in \mathcal{L}$ and every text for L . Gold, *Language Identification in the Limit*, 1967.

Gold's central negative result is simple to state. **Any class that contains every finite language and at least one infinite language cannot be identified in the limit from text.** The proof builds a single text on which the learner changes its mind infinitely often. Let $L_\infty = \{s_1, s_2, \dots\}$ be the infinite language. Present s_1 repeatedly. This is a text for the finite language $\{s_1\}$, so a successful learner must eventually guess $\{s_1\}$. Then present s_2 repeatedly until the learner guesses $\{s_1, s_2\}$, and so on. The resulting sequence contains every string of L_∞ , so it is a text for L_∞ , yet the learner's guesses never settle on L_∞ . Appendix A gives the details. The regular languages, and therefore the context-free languages, form such a class, so neither can be identified in the limit from positive examples alone. By contrast, Gold showed that much richer classes become identifiable if the learner also receives negative examples labeled as such.

The result was taken up in the debate over the **poverty of the stimulus**. Chomsky had argued that the evidence available to a child is too impoverished to determine the grammar of a language without strong innate constraints on the class of possible grammars. Chomsky, review of Skinner's *Verbal Behavior*, 1959. Gold's theorem offered a formal sense in which unconstrained learning from positive data is impossible. Its assumptions, however, are adversarial: the text may present strings in any order, with no regularities. If sentences are instead drawn at random from a probability distribution, the situation changes. James Horning showed in 1969 that probabilistic context-free grammars can be identified in a probabilistic sense from randomly drawn positive examples, and Dana Angluin extended the analysis of stochastic presentation in 1988. Distributional information can substitute for negative evidence: a string that should often appear but never does is evidence that it is not in the language.

A recent result reframes the question in a way that is directly relevant to language models. Jon Kleinberg and Sendhil Mullainathan ask for less than identification. The learner is shown a text for an unknown language from a countable collection, and it succeeds at **generation in the limit** if, after some finite time, every string it outputs is a new, previously unseen string of the target language. They prove that there is an agent that generates in the limit for *every* countable collection of candidate languages, in stark contrast to the impossibility of identification. Kleinberg and Mullainathan, *Language Generation in the Limit*, 2024. Producing valid new sentences is much easier than naming the language. The price is breadth: the agent's successful strategy retreats to safe subsets of the language, and later work studies the tension between generating only valid strings and generating all of them. The same tension appears in practice as the tradeoff between hallucination and mode collapse.

Form, meaning, and statistics

Chomsky's 1957 *Syntactic Structures* contains the most famous argument against statistical approaches to language. The sentences "Colorless green ideas sleep furiously" and "Furiously sleep ideas green colorless"

had presumably never occurred in any text, so, Chomsky argued, any statistical model of grammaticality would treat them as equally remote from English, although only the first is grammatical. Fernando Pereira answered forty years later with a calculation: a simple aggregate bigram model, which groups words into classes and was trained on newspaper text, assigns the grammatical sentence about 200,000 times the probability of its reversal. Pereira, *Formal Grammar and Information Theory: Together Again?*, 2000. A model that shares statistical strength across similar words can assign sensible probabilities to sentences it has never seen. This is the central idea of Weeks 2 and 3, and Peter Norvig’s essay on the “two cultures” of language science is a readable account of the debate. Norvig, *On Chomsky and the Two Cultures of Statistical Learning*.

Grammaticality is not meaning, however. Claude Shannon’s theory of communication deliberately set meaning aside: messages frequently have meaning, but “these semantic aspects of communication are irrelevant to the engineering problem.” Shannon, *A Mathematical Theory of Communication*, 1948. Entropy measures uncertainty about which message is sent, not what it says. A language model inherits this stance: its training objective is predictive, and it is rewarded for assigning high probability to observed text whether or not that text is true.

Emily Bender and Alexander Koller turned this into an argument that meaning cannot be learned from form alone. In their thought experiment, two people stranded on separate islands communicate through an underwater cable. A hyperintelligent octopus taps the cable, learns the statistical patterns of their messages, and eventually impersonates one of them convincingly. Then one of the islanders is chased by an angry bear, grabs a couple of sticks, and asks the other for advice on defending herself. The octopus cannot help, because nothing in the patterns of the messages connects words to the objects and situations they are about. Bender and Koller, *Climbing towards NLU*, 2020. This is a version of the older symbol grounding problem: how can the symbols of a formal system mean anything to the system itself, rather than only to the people who interpret it? Harnad, *The Symbol Grounding Problem*, 1990.

The opposing view holds that much of meaning is relational. Steven Piantadosi and Felix Hill argue that the meaning of a concept consists in large part of its relations to other concepts, its conceptual role, and that a model trained on text can learn such relations even without reference to the world. Piantadosi and Hill, *Meaning without Reference in Large Language Models*, 2022. The question is not settled, and this course does not settle it. It does suggest a more precise formulation. A distribution over text is a very rich object: its conditional structure encodes grammar, facts stated in text, and the patterns of reasoning people write down. The question is what information about the world, and about communicative intent, is recoverable from that distribution, and what additional information comes from the later stages of training, in which humans demonstrate, compare, and check.

A final result shows that even perfect statistical modeling has consequences one might not want. Adam Tauman Kalai and Santosh Vempala consider arbitrary facts, such as which paper a person wrote in a given year, whose truth cannot be inferred from patterns in other facts. They prove that a language model that is calibrated, in the sense that its probabilities match frequencies, must generate false statements of this kind at a rate roughly equal to the fraction of such facts that appear exactly once in its training data, a Good-Turing estimate of the unseen mass. Kalai and Vempala, *Calibrated Language Models Must Hallucinate*, 2024. Hallucination is not only an engineering defect; for this class of facts it is a statistical consequence of the pretraining objective, and reducing it requires something beyond pretraining, such as the post-training of Week 7 or retrieval from an external source.

5. Terminology and basics

This section fixes the vocabulary used for the rest of the course and reviews the basic mathematics. It is a map, not a course: each topic is developed in full in the supplementary material listed in Section 6.

AI, machine learning, and deep learning

Artificial intelligence is the study of computational systems that perform tasks associated with intelligence: perception, reasoning, language, planning, and action. Its methods include search (exploring a space of possibilities, as in game playing and planning), logic (representing knowledge and deriving consequences), probabilistic reasoning (representing uncertainty with probability distributions and updating them with evidence), and learning. A practical system usually combines several of these: a chat assistant is a learned model embedded in a program that manages conversation, may search over candidate answers, and may call tools.

Machine learning is the study of procedures that improve with experience. Tom Mitchell’s definition is still the most precise short one: a program learns from experience E with respect to a class of tasks T and a performance measure P if its performance on tasks in T , as measured by P , improves with E . For a language model, T might be predicting the next token of web text, E a corpus of documents, and P the average log probability assigned to held-out text.

Deep learning is machine learning with models built as compositions of many trainable transformations, usually neural networks, so that the model learns intermediate representations of its input along with its predictions. The depth is the length of the composition.

Statistics shares many of these methods: estimation, uncertainty, experimental design, and prediction. Neither field owns inference or prediction; the difference is mostly one of emphasis, with statistics making the data-generating assumptions explicit and machine learning emphasizing flexible models and computation.

Sources of learning signal

Learning procedures differ in where their training signal comes from.

- **Supervised learning** uses examples of inputs paired with targets, (x_i, y_i) : images with labels, or instructions with demonstrated answers.
- **Unsupervised learning** receives inputs without targets and models their structure: a density, clusters, or a low-dimensional representation.
- **Self-supervised learning** constructs targets from the data itself. A language model predicts each token from the ones before it; a masked model predicts hidden tokens from the visible ones. The provenance of the target differs from supervised learning, but the training calculation is an ordinary supervised loss.
- **Reinforcement learning** learns from rewards obtained by acting. The learner’s actions affect what it observes next, and rewards may arrive long after the actions that earned them.

These categories describe the source of the signal, not the architecture. A single Transformer is pretrained with self-supervision, finetuned with supervision, and adjusted with rewards, as the course will describe week by week.

Models, algorithms, and parameters

A **model** is a mathematical object used to represent a relationship or a distribution. A **learning algorithm** maps data to a fitted model; a separate computation then evaluates the fitted model on new inputs. The **representation** determines what information about an input reaches the model: a bag of words discards order, a token sequence keeps it. An **architecture** specifies the computation and how parameters are shared. **Parameters** are the quantities adjusted by learning, such as the weights of a network. **Hyperparameters** govern the model family or the learning procedure, such as the learning rate, the model width, or a regularization strength, and are chosen during development.

For example, represent an email by a vector x of features, with label $y = 1$ for spam and $y = 0$ otherwise. Logistic regression models the conditional probability of spam as

$$q_w(1 \mid x) = \sigma(w^\top x), \quad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

The vector w holds the parameters, $w^\top x$ is called the **logit**, and a decision rule such as “flag as spam if $q_w(1 \mid x) > 1/2$ ” turns the probability into an action.

Risk, generalization, and evaluation

A **loss** measures the cost of a prediction. For probabilistic predictions the standard choice is the log loss $\ell(\theta; x, y) = -\log q_\theta(y \mid x)$, which penalizes assigning low probability to what actually happened. If examples are drawn from a population distribution p_* , the quantity we care about is the **population risk** $R(\theta) = \mathbb{E}_{(x,y) \sim p_*}[\ell(\theta; x, y)]$. We can only compute the **empirical risk** on the n training examples,

$$\widehat{R}(\theta) = \frac{1}{n} \sum_{i=1}^n \ell(\theta; x_i, y_i),$$

and **empirical risk minimization** chooses θ to make it small, often with a regularization term added.

Generalization is the relationship between the two. **Overfitting** occurs when the empirical risk falls without a corresponding fall in the population risk. The Occam bound of Section 3 is the simplest example of a generalization guarantee: with enough examples relative to the size of the hypothesis class, empirical and population performance are close.

In practice, generalization is estimated rather than proved. Training data fit the parameters; **validation data** guide development choices such as the architecture, learning rate, and stopping time; **test data** estimate performance once those choices are fixed. The separation concerns information flow, not filenames: a test set that has been used to choose a model has become part of development. For text, overlapping windows or duplicated passages can leak information across a split even when the split looks clean, so splits are made by document, and duplicates are removed.

Probability, likelihood, and inference

For independent examples, maximizing the likelihood of the data is the same as minimizing the total log loss. For a model of a distribution $q_\theta(x)$ fitted to samples from p_* , the expected log loss is the cross-entropy, so

$$\mathbb{E}_{x \sim p_*}[-\log q_\theta(x)] = H(p_*) + D_{\text{KL}}(p_* \parallel q_\theta).$$

The entropy $H(p_*)$ does not depend on θ , so in the limit of infinite data maximum likelihood minimizes the KL divergence from the true distribution to the model. This one identity connects the language-model training objective to the compression view of Section 3 and to the perplexity evaluations of Week 2.

A **generative** classifier models the joint distribution of inputs and labels, often as $q(x, y) = q(x \mid y)q(y)$, and classifies with Bayes' rule,

$$q(y \mid x) = \frac{q(x \mid y)q(y)}{\sum_{y'} q(x \mid y')q(y')}.$$

A **discriminative** classifier models $q(y \mid x)$ directly, as logistic regression does. Ng and Jordan, 2001. The word "generative" in generative AI is broader: it refers to models from which new objects can be sampled. A language model is generative in this sense even though each of its conditionals is a classifier over the next token.

The word **inference** is badly overloaded, and it is worth separating its meanings once:

Term	Meaning	Example
Statistical inference	Reasoning from data to unknown quantities, including their uncertainty	A confidence interval for a model's accuracy
Probabilistic inference	Computing a quantity within a specified model	The posterior probability of spam given the words of an email
Inference (deployment)	Running a trained model	Generating a response to a prompt

Optimization and backpropagation

Most models in this course are fitted by gradient methods. **Gradient descent** repeatedly moves the parameters against the gradient of the loss, $\theta \leftarrow \theta - \eta \nabla \widehat{R}(\theta)$, with a learning rate $\eta > 0$. **Stochastic gradient descent** replaces the full gradient with an unbiased estimate computed from a small random subset of examples, a **minibatch**, trading noise for much cheaper steps. Variants such as momentum, Adam, and AdamW adapt the step using the history of gradients; Week 4 examines them.

For logistic regression with log loss, the gradient on one example is $(\sigma(w^\top x) - y)x$: the prediction error times the input. Starting from $w = (0, 0)$ with $x = (2, 1)$ and $y = 1$, the predicted probability is $1/2$, the loss is $\ln 2 \approx 0.693$, and the gradient is $-\frac{1}{2}(2, 1)$. One step with $\eta = 0.1$ gives $w = (0.1, 0.05)$, and the predicted probability of the correct label rises to $\sigma(0.25) \approx 0.562$.

Backpropagation computes the gradient of a scalar loss with respect to all parameters of a composed computation by applying the chain rule backwards through the computation graph. It is reverse-mode automatic differentiation, and it has a remarkable complexity property: the gradient of a function with respect to all of its inputs can be computed with only a small constant factor more arithmetic than evaluating the function once. Baur and Strassen, 1983. This cheap gradient principle is why networks

with billions of parameters can be trained at all. Backpropagation computes derivatives; the **optimizer** decides how to use them. Rumelhart, Hinton, and Williams, 1986.

Neural networks

A feedforward neural network composes affine maps with fixed nonlinear functions: a layer computes $h \mapsto \phi(Wh + b)$, where W and b are parameters and ϕ is applied coordinatewise, for example $\phi(z) = \max(0, z)$. Without the nonlinearity a stack of layers would collapse to a single affine map. With it, networks with one hidden layer of sufficient width can approximate any continuous function on a compact set to any accuracy. Cybenko, 1989; Hornik, 1991.

The universal approximation theorem is a statement about expressivity only, and a TCS reader should resist over-reading it. It says nothing about how large the network must be, how many examples are needed to find a good approximation, or whether gradient descent will find it. Learning parity functions of a few hidden bits from random examples, for instance, is easy to express and notoriously hard for gradient methods. The success of deep learning is an empirical fact about how expressivity, optimization, and generalization interact on natural data, and the theory explaining it is incomplete. LeCun, Bengio, and Hinton, 2015.

Architectures differ in how they share parameters, which encodes an assumption about the data. A convolutional network reuses the same filters at every position of an image, assuming that local patterns matter wherever they occur. A recurrent network applies the same update at every position of a sequence, carrying a hidden state that summarizes the past. A Transformer lets every position of a sequence read from every earlier position through attention, with the same parameters at every position. Weeks 2 and 3 develop the last two in detail.

From the basics to language models

A language model applies these ingredients to the distribution over strings of Section 1. Each conditional $q_\theta(x_t \mid x_{<t})$ is a classifier over the vocabulary, computed by a network from the prefix; training minimizes the log loss of observed next tokens, which is self-supervised; and generation samples one token at a time, feeding each back as input. The lifecycle of a modern model has four stages, each with its own source of learning signal:

Stage	Signal	What changes	Weeks
Pretraining	Next tokens of a large text corpus	All parameters, from a random start	2-4
Supervised finetuning	Demonstrations of desired behavior	Parameters, from the pre-trained model	5, 7
Preference and RL post-training	Comparisons and rewards	Parameters, relative to a reference model	6, 7
Inference	None: prompts, retrieved text, and tool outputs condition a fixed model	Nothing in the parameters	5, 8

The last row is a distinction worth keeping in mind from the start. A model can appear to learn during a conversation, because a fact placed in its context changes its predictions. That is conditioning, not learning in the sense of this section: nothing persists once the context is gone.

6. Supplementary material

The basics of Section 5 are developed in full elsewhere. The table lists where. "Companion" refers to the course's *AI and Machine Learning* companion; the module chapters refer to the Background Notes on Machine Learning and AI.

Topic	Companion	Background Notes	Textbook
Terminology; the objects being learned	Chapter 1	Foundations 1	Russell and Norvig, Chapters 1-2
Linear algebra and calculus	Chapter 2	Foundations 2, 3	Goodfellow et al., Chapters 2, 4

Topic	Companion	Background Notes	Textbook
Probability, likelihood, and Bayes' rule	Chapter 5	Foundations 4; ML 4, 5	Goodfellow et al., Chapter 3
Risk, generalization, and PAC learning	Chapter 3	Foundations 5; ML 7	Shalev-Shwartz and Ben-David, Chapters 2-6
Information theory	Chapter 6	Foundations 5	Cover and Thomas, Chapter 2
Evaluation and data leakage	Chapter 7	ML 6	
Neural networks and back-propagation	Chapter 10	DL 1; Foundations 3	Goodfellow et al., Chapter 6
Optimization and regularization	Chapter 11	DL 3, 5	Goodfellow et al., Chapters 7-8
Search, logic, and probabilistic reasoning		AI 1-9	Russell and Norvig, Parts II-IV
Numerical computing with arrays		Foundations 6	

For reading in the order of this lecture: Turing's 1950 paper is short and still rewarding; Valiant's 1984 paper introduces the PAC model in eight pages; Chapters 2 and 3 of Shalev-Shwartz and Ben-David develop the learning theory; and the companion's Chapters 1, 3, 5, and 7 cover the rest of Section 5 at the level assumed in Weeks 2 through 4.

7. Exercises

Exercise 1: The Occam bound

1. A consistent learner searches a class of 2^{20} hypotheses. How many examples guarantee error at most 0.05 with probability at least 0.99? How does the answer change if the class has 2^{40} hypotheses, and if the required error is halved?
2. Let $\mathcal{H}$ be all Boolean functions on n bits. Evaluate the Occam bound and explain its relation to the no-free-lunch example of Section 3.
3. The derivation used $(1 - \epsilon)^m \leq e^{-\epsilon m}$ and a union bound. Which step uses the assumption that training and test inputs come from the same distribution?

Exercise 2: Identification and generation in the limit

Let $\Sigma = \{a\}$, so every language is a set of strings a^k .

1. Show that the class of all finite languages over Σ is identifiable in the limit from text. (Hint: guess the set of strings seen so far.)
2. Show that adding the single infinite language a^* makes the class non-identifiable, by carrying out the construction of Appendix A for your learner from part 1.
3. Now consider the class of co-finite languages over Σ , each of the form a^* minus a finite set. Show that this class is not identifiable in the limit from text. (Hint: adapt Appendix A, alternating between texts for $a^* \setminus \{a^k\}$ with ever larger k , and then showing a^k .)
4. For the co-finite class, describe a strategy that *generates* in the limit: after finitely many steps, every string it outputs belongs to the target language and has not yet been shown. Why did Kleinberg and Mullainathan assume that every candidate language is infinite?

Exercise 3: Prediction and compression

Let a source emit symbols $\{A, B, C\}$ with probabilities $p = (1/2, 1/4, 1/4)$.

1. Compute the entropy $H(p)$ in bits and give a prefix code that achieves it.

2. A model uses the uniform distribution $q = (1/3, 1/3, 1/3)$. Compute the cross-entropy $H(p, q)$ and the KL divergence $D_{\text{KL}}(p\|q)$ in bits, and interpret the difference as extra code length.
3. Explain why minimizing the average log loss of a language model on held-out text is the same as minimizing the length of an ideal code for that text built from the model.

Exercise 4: Logistic regression and generalization

Use $q_w(y = 1 \mid x) = \sigma(w^\top x)$ with log loss.

1. Starting from $w = (0, 0)$, take one gradient step with $\eta = 0.1$ on the example $x = (2, 1)$, $y = 1$. Report the loss before and after.
2. Evaluate the loss on a second example $x' = (-2, 1)$, $y' = 1$ before and after the same step. Explain why an update that helps one example does not demonstrate generalization.
3. Classify each of the following as a parameter, a hyperparameter, or an evaluation choice: w ; the learning rate; the number of steps; holding out every email from particular senders.

Exercise 5: Bayes' rule

A model assigns prior probability 0.2 to spam. The word "prize" appears in spam with probability 0.6 and in other email with probability 0.05.

1. Compute the probability of spam given that an email contains "prize".
2. Which quantities does a generative classifier estimate here, and which does a discriminative classifier estimate instead?
3. Give one example each of statistical inference, probabilistic inference, and deployment inference in this setting.

Exercise 6: Sources of learning signal

For each situation, identify the task, the source of the learning signal, and what changes in the model.

1. A Transformer predicts masked words from the surrounding text.
2. A pretrained model is finetuned on instruction-answer pairs written by people.
3. A fixed model receives retrieved documents before answering a question.
4. A model samples programs, receives unit-test results, and updates its parameters to make passing programs more likely.

Exercise 7: Meaning and its evidence

1. Compare "Mira admires Arun" with "Arun admires Mira." What does a bag-of-words representation lose? Give a relation interpreting "admires" under which one sentence is true and the other false.
2. Explain why "Some succeeded" suggests that not all succeeded without entailing it. Give a continuation that cancels the suggestion.
3. "Increase" and "decrease" have similar distributions. Describe a prediction task on which a model that confuses them would fail.
4. Summarize, in a paragraph, the strongest version of the argument that a model trained only on text cannot learn meaning, and the strongest reply. What experiment, if any, could distinguish the two positions?

Exercise 8: Definitions of intelligence

1. Why is the Legg-Hutter measure incomputable? Name the two separate sources of incomputability.
2. Give an example of a system that would score well on a fixed benchmark of tasks but poorly under Chollet's skill-acquisition definition, and explain the difference.

3. Turing’s prediction for the year 2000 was stated in terms of storage (10^9 bits) rather than computation. What quantities would you use today to describe the resources of a language model, and why?

Appendix A. Gold’s theorem

Fix a countable alphabet of strings $s_1, s_2, \dots$ and a class $\mathcal{L}$ that contains every finite language and an infinite language $L_\infty = \{s_1, s_2, \dots\}$. A learner is a function M from finite sequences of strings to guesses. Suppose, for contradiction, that M identifies every language of $\mathcal{L}$ in the limit from text.

Construct a sequence in stages. At stage $k \geq 1$, the sequence so far is τ_{k-1} , which contains exactly the strings $s_1, \dots, s_{k-1}$ (with τ_0 empty). Extend it by repeating s_k . The infinite sequence $\tau_{k-1}s_k s_k s_k \dots$ is a text for the finite language $F_k = \{s_1, \dots, s_k\}$, which belongs to $\mathcal{L}$. Since M identifies F_k in the limit, there is a finite number of repetitions after which M guesses F_k . Let τ_k be τ_{k-1} followed by that many copies of s_k .

The limit $\tau = \lim_k \tau_k$ is an infinite sequence that contains every s_k and nothing else, so it is a text for L_∞ . On this text, M guesses F_1 at the end of τ_1 , F_2 at the end of τ_2 , and so on. These guesses are all different and none of them is L_∞ , so M ’s guesses never converge to a grammar for L_∞ . This contradicts the assumption that M identifies L_∞ . $\square$

The argument needs only that the learner must succeed on every text, including adversarially ordered ones, and that the class is rich enough to contain all the finite approximations of an infinite language. Both assumptions are what probabilistic presentation and restricted classes relax. Angluin (1980) characterized exactly which classes are identifiable from text, in terms of finite ”tell-tale” subsets that distinguish each language from its subsets in the class.

References and further reading

Intelligence

- Alan M. Turing (1950), **Computing Machinery and Intelligence**. Paper. The imitation game, objections to machine intelligence, and learning machines.
- John McCarthy, Marvin Minsky, Nathaniel Rochester, and Claude Shannon (1955), **A Proposal for the Dartmouth Summer Research Project on Artificial Intelligence**. Proposal.
- Allen Newell and Herbert A. Simon (1976), **Computer Science as Empirical Inquiry: Symbols and Search**. Paper. The physical symbol system hypothesis.
- Warren McCulloch and Walter Pitts (1943), **A Logical Calculus of the Ideas Immanent in Nervous Activity**. Paper.
- Frank Rosenblatt (1958), **The Perceptron**. Paper.
- Stuart Russell and Peter Norvig, **Artificial Intelligence: A Modern Approach**, 4th edition (2020). Book site. Definitions of AI and the rational-agent view.
- Shane Legg and Marcus Hutter (2007), **Universal Intelligence: A Definition of Machine Intelligence**. Paper.
- François Chollet (2019), **On the Measure of Intelligence**. Paper.
- John R. Searle (1980), **Minds, Brains, and Programs**. Paper. The Chinese room.
- Richard Sutton (2019), **The Bitter Lesson**. Essay.

Learning

- David Hume (1748), **An Enquiry Concerning Human Understanding**. Text. Sections IV-V on induction.
- David H. Wolpert (1996), **The Lack of A Priori Distinctions Between Learning Algorithms**. Paper. No free lunch.
- Leslie G. Valiant (1984), **A Theory of the Learnable**. Paper. The PAC model.

- **Shai Shalev-Shwartz and Shai Ben-David (2014), Understanding Machine Learning: From Theory to Algorithms.** Book. Chapters 2-6 for PAC learning, uniform convergence, and VC dimension.
- **Michael Kearns and Leslie G. Valiant (1994), Cryptographic Limitations on Learning Boolean Formulae and Finite Automata.** Paper.
- **Dana Angluin (1987), Learning Regular Sets from Queries and Counterexamples.** Paper. The L^* algorithm.
- **Ray J. Solomonoff (1964), A Formal Theory of Inductive Inference, Part I.** Paper.
- **Jorma Rissanen (1978), Modeling by Shortest Data Description.** Paper. Minimum description length.

Language and meaning

- **Pāṇini, Aṣṭādhyāyī.** Śrīśa Chandra Vasu's English translation, volume I (1891).
- **Aristotle, On Interpretation.** E. M. Edghill's translation, chapters 1-4.
- **Gottlob Frege (1892), On Sense and Reference.** English translation.
- **Ludwig Wittgenstein (1953), Philosophical Investigations.** Published edition, especially sections 2, 23, and 43.
- **H. P. Grice (1975), Logic and Conversation.** Chapter.
- **Zellig S. Harris (1954), Distributional Structure.** Paper.
- **J. R. Firth (1957), A Synopsis of Linguistic Theory, 1930-1955.** Essay.
- **Noam Chomsky (1956), Three Models for the Description of Language.** Paper. Also *Syntactic Structures* (1957), Chapter 2, for "colorless green ideas".
- **Noam Chomsky (1959), Review of B. F. Skinner's Verbal Behavior.** Paper.
- **E. Mark Gold (1967), Language Identification in the Limit.** Paper.
- **Jon Kleinberg and Sendhil Mullainathan (2024), Language Generation in the Limit.** Paper.
- **Claude E. Shannon (1948), A Mathematical Theory of Communication.** Paper.
- **Fernando Pereira (2000), Formal Grammar and Information Theory: Together Again?** Paper.
- **Peter Norvig, On Chomsky and the Two Cultures of Statistical Learning.** Essay.
- **Emily M. Bender and Alexander Koller (2020), Climbing towards NLU: On Meaning, Form, and Understanding in the Age of Data.** Paper.
- **Stevan Harnad (1990), The Symbol Grounding Problem.** Paper.
- **Steven T. Piantadosi and Felix Hill (2022), Meaning without Reference in Large Language Models.** Paper.
- **Adam Tauman Kalai and Santosh S. Vempala (2024), Calibrated Language Models Must Hallucinate.** Paper.

Basics

- **Andrew Y. Ng and Michael I. Jordan (2001), On Discriminative vs. Generative Classifiers.** Paper.
- **Walter Baur and Volker Strassen (1983), The Complexity of Partial Derivatives.** Paper.
- **David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams (1986), Learning Representations by Back-Propagating Errors.** Paper.
- **George Cybenko (1989), Approximation by Superpositions of a Sigmoidal Function.** Paper. **Kurt Hornik (1991), Approximation Capabilities of Multilayer Feedforward Networks.** Paper.
- **Yann LeCun, Yoshua Bengio, and Geoffrey Hinton (2015), Deep Learning.** Review.
- **Ian Goodfellow, Yoshua Bengio, and Aaron Courville (2016), Deep Learning.** Book.
- **Thomas M. Cover and Joy A. Thomas, Elements of Information Theory, 2nd edition (2006).** Chapters 2 and 5 on entropy and coding.

- **The course's AI and Machine Learning companion.** Posted on the course page. Chapters 1-3, 5-7, and 10-11 develop the basics of Section 5 in full.